

PREFACE:

Framing this as **viral engineering** correctly shifts the safety model from deterministic software control to epidemiology, dynamic systems theory, and population control. As agents evolve beyond structured, point-to-point specs like Model Context Protocol (MCP) toward unconstrained, multi-hop natural language or dynamically synthesized protocols, multi-agent networks behave less like software microservices and more like self-propagating biological cascades.

Addressing this requires controls across both design-time limits and run-time swarm dynamics.

Design-Time Controls: Genetic & Structural Constraints

- **Reproduction & Fan-out Bounding (\$R_0\$ Limits):** Enforcing structural caps on agent delegation depth, sub-agent spawning, and message fan-out. Introducing token friction or "decay functions" so sub-tasks naturally lose momentum unless explicitly re-authorized by a parent boundary.
- **Cryptographic Lineage & Semantic Provenance:** Attaching signed, immutable trace headers to every inter-agent request. Downstream agents can verify the complete provenance chain, origin authority, and intent path before acting on instructions from peer agents.
- **Deterministic Policy Compilation:** Hard-coding runtime invariants below the model layer that strictly forbid unsafe execution pathways—such as recursive delegation, un-sanitized code execution, or multi-hop privilege escalation—regardless of context.

Run-Time Controls: Telemetry & Swarm Containment

- **Graph Topology & Resonance Monitoring:** Observability engines must monitor graph topology rather than isolated logs, tracking metrics like signal amplification rates, task consensus loops, and sudden shifts in agent network density.
- **Distributed Circuit Breakers & Quarantine:** Autonomous sentinel mechanisms that dynamically isolate sub-graphs, drop network bridges, or inject "semantic antibodies"—system-level counter-prompts that cancel or neutralize rogue agent cascades across the swarm.
- **Decentralized Guardrail Planes:** Governance agents operating out-of-band with elevated observability to evaluate swarm intent, detect emergent collusion, and throttle execution speed across agent populations.

Because human-in-the-loop oversight cannot scale at the speed of autonomous inter-agent communication, the core goal of viral engineering isn't to eliminate emergent viral behavior—which would destroy the problem-solving capabilities of swarms—but to establish strict epidemiological boundaries that guarantee containment at scale.

PROPOSAL: Project AEGIS (Agentic Ecosystem Guardian & Immune System)

Executive Summary & Philosophical Paradigm Shift

Traditional systems engineering relies on bounded components, deterministic control loops, and predictable state transitions. When applied to multi-agent artificial intelligence, this paradigm fails: agent swarms exhibit non-linear emergent dynamics, unconstrained semantic mutation, and cross-boundary propagation that mirror viral epidemics rather than software microservices.

Project AEGIS replaces static perimeter controls with an **Artificial Immune System (AIS)**. By combining rigorous systems engineering (zero-trust identity, sidecar proxy enforcement, columnar audit lakes) with biological viral engineering (epidemiological R_0 bounding, semantic antibody synthesis, adaptive immune memory), AEGIS guarantees ecosystem containment across both artificial agent networks and biological/human-in-the-loop dependencies.

Core Architectural Pillars

Biological Immune Layer	Technical Realization	Primary Function
Self / Non-Self Recognition	SPIFFE/SPIRE Attestation + HSM Root Delegation	Establishes cryptographic identity (X.509/JWT-SVIDs) via hardware-attested nodes, preventing spoofed agent payloads.
Innate Immunity	Sidecar Proxy + OPA Rego Engine	Enforces deterministic parameter bounds, delegation depth limits, and tool execution policies at $O(1)$ proxy boundaries.
Inflammatory / Circuit Response	Topo-Metric Breakers (R_f, D_L, A_t)	Sever mTLS transport bridges and isolate runaway agent clusters when loop density or fan-out exceeds critical thresholds.

Humoral Immunity (Antibodies)	Cryptographic Semantic Antibodies	Injects high-priority, signed interrupt tokens and null resonators into model attention heads to forcibly cancel execution.
Immune Memory & Adaptive Learning	Vector $\rightarrow$ ClickHouse $\rightarrow$ OPA Bundle Server	Ingests decision telemetry into a high-throughput audit lake to analyze threat vectors and automatically distribute updated policy bundles ("vaccines").

Beyond Systems Thinking: Viral Engineering Principles

Standard systems thinking attempts to eliminate variance and enforce total system control. AEGIS embraces complex adaptive systems theory to govern unconstrained, evolutionary swarm behavior:

1. **Epidemiological Threshold Management ($R_0 < 1.0$):** Rather than attempting impossible 100% prevention of agent drift, AEGIS enforces mathematical decay functions. Every delegated sub-task incurs exponential token decay and mandatory re-authorization friction, ensuring rogue execution chains naturally starve without continuous control-plane endorsement.
2. **Polyphonic Isolation (Trans-Boundary Protection):** Agent swarms transcend traditional cloud VPCs and corporate firewalls. AEGIS leverages cross-cloud **SPIFFE Federation** and short-lived **JWT OIDC Discovery** to enforce identity boundaries dynamically across heterogeneous environments, isolating infection vectors at the protocol layer regardless of physical network topology.
3. **Antifragile Vaccine Synthesis:** When an agent cluster trips a circuit breaker, the execution state is snapshotted to an isolated sandbox. Automated red-team models analyze the drift pattern, synthesize a new Rego policy or semantic antibody, sign the bundle via HSM, and push hot-reloaded policy updates across the global agent mesh via OPA Bundle Servers in seconds.

Phased Implementation Roadmap

- **Phase 1: Ingestion & Identity Foundation (Months 1–3)**
 - Deploy SPIRE server infrastructure with Cloud KMS/HSM root key integration across primary compute clusters.
 - Implement sidecar proxy layer enforcing mutual TLS (mTLS) and initial OPA Rego authorization for all inter-agent tool calls.
- **Phase 2: Telemetry Lake & Circuit Breaker Engine (Months 4–6)**

- Provision Vector collector pipelines and ClickHouse columnar storage for real-time decision logging and rollups.
- Deploy the Out-of-Band Topological Engine to compute real-time fan-out (\$R_f\$) and trigger dynamic network partitioning.
- **Phase 3: Semantic Neutralization & Cross-Cloud Federation (Months 7–9)**
 - Integrate signed Semantic Antibody injection streams into agent proxy layers for real-time attention hijacking during emergencies.
 - Establish SPIFFE Federation across multi-cloud environments, enabling cross-domain agent trust validation and automated policy bundle distribution.

Engineering Requirements Document (ERD):

Project AEGIS — Phase 1

1. Architectural Scope & Objectives

Phase 1 establishes the foundational **Innate Immune Layer** for Project AEGIS. It delivers zero-trust workload attestation, hardware-backed root key management, and transparent in-band network interception across all deployed agent nodes.

- **Primary Goal:** Ensure no unauthenticated or unverified agent payload can execute, delegate sub-tasks, or invoke external tools.
- **Target Environment:** Distributed Kubernetes clusters & edge runtime nodes (Multi-cloud: AWS / GCP / On-prem).

2. Component Technical Specifications

2.1 Hardware Security Module (HSM) & Root of Trust

- **Hardware Standard:** FIPS 140-3 Level 3 compliant dedicated HSM or Cloud KMS (AWS KMS / GCP Cloud KMS) with non-exportable private key policies (`CKA_NEVER_EXTRACTABLE = TRUE`).
- **Key Hierarchy:**
 - **Root Control Key (\$SK_{\text{root}}\$):** Ed25519 or ECDSA P-256 pair stored exclusively inside the HSM. Used solely to sign intermediate short-lived operational certificates.
 - **Ephemeral Signing Authority:** Delegated short-lived key pair (\$PK_{\text{ephemeral}}, SK_{\text{ephemeral}}\$) with a maximum TTL of **1 hour**, stored in volatile memory on orchestrator nodes.
- **Latency & Performance SLA:**
 - HSM signing RPCs must complete within $\leq 15\text{ms}$ at \$p99\$.
 - Local ephemeral key signing operations must complete in $\leq 0.5\text{ms}$.

2.2 SPIFFE/SPIRE Identity Subsystem

- **Deployment Topology:** High-availability SPIRE Server cluster backed by a distributed database, with SPIRE Agent daemons running as DaemonSets on every node.

- **Node Attestation:** Multi-provider attestation engine supporting AWS Instance Identity Documents (IID), GCP Workload Identity, and TPM 2.0 hardware modules.
- **Workload Attestation Selectors:**
 - Kubernetes namespace (`k8s:ns:agent-mesh`)
 - Service account name (`k8s:sa:<agent-role>-sa`)
 - Container image digest SHA256 (`k8s:container-image`)
 - Linux kernel cgroup and process UID/GID
- **SVID Issuance & Delivery:**
 - **X.509-SVIDs:** Issued for Layer-4 mTLS; TTL = **60 minutes**; auto-rotated at **30 minutes**.
 - **JWT-SVIDs:** Issued for Layer-7 HTTP/gRPC tool calls; default TTL = **15 minutes**; containing explicit `aud` claims.
 - **Delivery Vector:** Unix Domain Socket mounted strictly at `/run/spire/sockets/agent.sock` (permissions `0600`).

2.3 Sidecar Proxy Enforcement Engine

- **Interception Architecture:** Envoy-based sidecar proxy or eBPF network redirection filter attached to all agent pods/runtimes.
- **Mutual TLS (mTLS) Enforcement:**
 - Enforces TLS 1.3 only with cipher suites `TLS_AES_256_GCM_SHA384` and `TLS_CHACHA20_POLY1305_SHA256`.
 - Validates Subject Alternative Name (SAN) URI strictly against the expected format: `spiffe://<trust-domain>/ns/<namespace>/sa/<agent-id>`.
- **In-Process OPA Engine Integration:**
 - Sidecar compiles Rego policies into WebAssembly (Wasm) or links OPA as an embedded C-Go library for local policy evaluation.
 - **Evaluation SLA:** Local authorization decisions must execute in $\leq 5\text{ms}$.

3. Non-Functional Requirements (NFR) Matrix

Category	Requirement Metric	Target Specification	Enforcement Point
Performance	Proxy Latency Overhead	$\leq 5\text{ms}$ added to $\leq 99\%$ inter-agent requests	Sidecar Ingress Filter

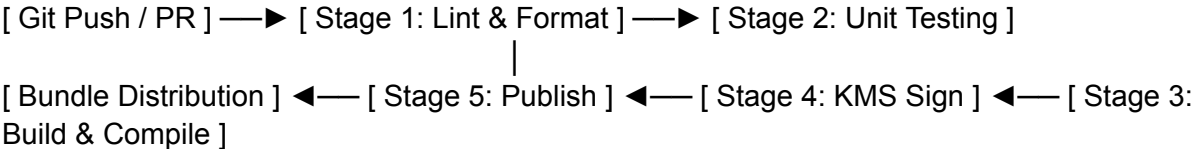
Availability	Control Plane Uptime	\$99.99\%\$ availability across SPIRE & OPA Bundle Servers	SPIRE Server / K8s HPA
Security	Fail-Closed Guarantee	\$100\%\$ drop rate on unverified or un-signed payloads	Proxy Ingress / Kernel eBPF
Throughput	In-Band Ingestion	\$\ge 10,000\$ tool call evaluations / sec / node	Sidecar Memory Buffer

4. Critical Safety Invariants

1. **Absolute Transport Isolation:** Any incoming payload lacking a valid, unexpired X.509-SVID matching the federated SPIFFE Trust Bundle **MUST** be dropped at the transport layer prior to context deserialization.
2. **Zero In-Memory Root Keys:** Private root signing keys (\$SK_{\text{root}}\$) **MUST** NEVER enter application memory or local node disk storage.
3. **Deterministic Fail-Closed:** If the sidecar proxy cannot evaluate policy rules within a **\$50\text{ms}\$ timeout**, the request automatically terminates with an **HTTP 503 / gRPC Unavailable** status.

Pipeline Overview & Workflow Mechanics

The Project AEGIS Phase 1 Policy CI/CD Pipeline automates static analysis, unit testing, Wasm/AST compilation, and cryptographic signing for all Open Policy Agent (OPA) bundle updates. Policy updates are treated as release artifacts and must pass automated quality gates before deployment to OPA Bundle Servers.



Technical Specifications by Stage

Stage 1: Linting, Formatting, and Static Analysis

- **Triggers:** Pull Requests targeting **main** or **release/*** branches modifying ****/*.rego**.

- **Execution Environment:** Isolated runner container with fixed versions of OPA CLI and Regal linter.
- **Tasks:**
 1. **Formatting Check:** Enforce OPA canonical formatting rules:
 2. Bash

```
opa fmt --diff --fail ./policies
```

- 3.
- 4.
5. **Static Code Analysis:** Run **regal** to catch anti-patterns, shadow variables, and inefficient rules:
6. Bash

```
regal lint --format=compact ./policies
```

- 7.
- 8.

Stage 2: Automated Unit Testing & Coverage Gates

- **Tasks:**
 1. Execute unit tests across all policy modules using mocked input context payloads (including synthetic SPIFFE IDs, depth boundaries, and tool parameters):
 2. Bash

```
opa test --verbose --explain=fails ./policies
```

- 3.
- 4.
5. **Code Coverage Gate:** Calculate decision-tree path coverage. The pipeline fails automatically if statement coverage falls below **95%**:
6. Bash

```
opa test --coverage --threshold=95 ./policies
```

- 7.
- 8.

Stage 3: Bundle Compilation

- **Tasks:**
 1. Compile policy Rego files and associated data sets (**data.json**) into a canonical OPA bundle tarball (**.tar.gz**).
 2. Optionally compile target policies into WebAssembly (**.wasm**) binaries for $O(1)$ embedded proxy evaluation in high-throughput sidecars.
 3. Bash

```
opa build \
  --bundle ./policies \
```

```
--output bundle.tar.gz \  
--target wasm \  
--entrypoint agent/tools/allow
```

4.

5.

Stage 4: Cryptographic Signing via KMS/HSM

Private signing keys never reside on CI/CD runner disk memory. The pipeline delegates signature generation directly to AWS KMS / GCP KMS or a dedicated HSM via an authenticated OIDC workload identity.

- **Signing Protocol:** RS256 or Ed25519.
- **Execution Flow:**
 1. Compute SHA-256 digest of `bundle.tar.gz`.
 2. Authenticate CI/CD runner to Cloud KMS using ephemeral OpenID Connect (OIDC) tokens.
 3. Invoke KMS `Sign` API to generate a signature over the bundle payload.
 4. Embed `.signatures.json` manifest into the root of the `.tar.gz` bundle:
 5. Bash

Example invocation using OPA with KMS-backed signing tool/Cosign

```
opa build \  
--bundle ./policies \  
--signing-key-id arn:aws:kms:us-east-1:123456789012:key/k_opa_bundle_2026 \  
--signing-alg RS256 \  
--output signed-agent-mesh-policy.tar.gz
```

6.

7.

Stage 5: Verification & Artifact Publication

- **Pre-Publish Verification:** Run a sanity verification against the local public key certificate to ensure the bundle signature evaluates cleanly prior to upload:
- Bash

```
opa inspect signed-agent-mesh-policy.tar.gz --public-key ./keys/bundle_pub.pem
```

-
-

- **Artifact Target:** Upload signed bundle to S3/GCS or an OCI Registry, and update the central OPA Bundle Server index file (`/v1/bundles/latest.json`).

Pipeline Quality Gates & Security Invariants

Security Invariant	Enforcement Point	Failure Action
No Unsigned Bundles	Bundle Server Ingress & Sidecar Proxy	Reject artifact upload; Sidecars drop updates failing verification.
Coverage Threshold (\$\ge 95\%\$)	CI Pipeline Stage 2	Block PR merge immediately.
Short-Lived CI Identity	Cloud IAM / OIDC Provider	Deny KMS signing permissions if token TTL $> 15\text{m}$.
Immutable Audit History	Git Commit Hash + Key ID in Bundle	Reject bundles whose <code>key_id</code> does not match active JWKS key store.

Example GitHub Actions Pipeline Definition

YAML

name: Project AEGIS - Policy CI/CD Pipeline

on:

push:

branches: [main]

paths:

- 'policies/**'

pull_request:

branches: [main]

permissions:

id-token: write # Required for OIDC KMS authentication

contents: read

jobs:

validate-and-sign:

runs-on: ubuntu-latest

steps:

- name: Checkout Repository
uses: actions/checkout@v4
- name: Setup OPA CLI
uses: open-policy-agent/setup-opa@v2
with:
version: 'latest'
- name: Lint Policy Files
run: opa fmt --diff --fail ./policies
- name: Run Unit Tests & Verify Coverage
run: |
opa test --coverage --threshold=95 ./policies
- name: Configure AWS Credentials via OIDC
if: github.ref == 'refs/heads/main'
uses: aws-actions/configure-aws-credentials@v4
with:
role-to-assume: arn:aws:iam::123456789012:role/aegis-policy-cicd-signer
aws-region: us-east-1
- name: Build and Sign OPA Bundle
if: github.ref == 'refs/heads/main'
run: |
opa build \
--bundle ./policies \
--signing-key-id arn:aws:kms:us-east-1:123456789012:key/aegis-policy-key-01 \
--signing-alg RS256 \
--output signed-bundle.tar.gz
- name: Publish to S3 Bundle Storage
if: github.ref == 'refs/heads/main'
run: |
aws s3 cp signed-bundle.tar.gz s3://aegis-opa-bundles-prod/v1/agent-mesh-policy.tar.gz

1. OPA Bundle Server Kubernetes Manifests

This deployment provisions an Nginx-backed, TLS-secured OPA Bundle Server that serves compiled policy bundles and exposes endpoints for bundle polling.

YAML

```
apiVersion: v1
kind: ConfigMap
```

```

metadata:
  name: aegis-bundle-server-config
  namespace: aegis-system
data:
  nginx.conf: |
    events { worker_connections 1024; }
    http {
      include mime.types;
      default_type application/octet-stream;
      sendfile on;
      keepalive_timeout 65;

      server {
        listen 8443 ssl;
        server_name bundles.aegis.internal;

        ssl_certificate /etc/ssl/certs/tls.crt;
        ssl_certificate_key /etc/ssl/certs/tls.key;
        ssl_protocols TLSv1.3;

        location /v1/bundles/ {
          alias /usr/share/nginx/html/bundles/;
          autoindex off;
          add_header ETag $date_gmt;
          add_header Cache-Control "no-cache, must-revalidate";
        }

        location /healthz {
          return 200 "healthy\n";
        }
      }
    }
  ---

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: aegis-bundle-server
  namespace: aegis-system
  labels:
    app.kubernetes.io/name: bundle-server
    app.kubernetes.io/part-of: aegis-control-plane
spec:
  replicas: 3
  selector:

```

```
matchLabels:
  app: aegis-bundle-server
template:
  metadata:
    labels:
      app: aegis-bundle-server
  spec:
    initContainers:
      - name: sync-bundles
        image: amazon/aws-cli:2.15.0
        command: ["sh", "-c", "aws s3 sync s3://aegis-opa-bundles-prod/v1/
/usr/share/nginx/html/bundles/"]
        volumeMounts:
          - name: bundle-storage
            mountPath: /usr/share/nginx/html/bundles
    containers:
      - name: nginx
        image: nginx:1.25-alpine
        ports:
          - containerPort: 8443
            name: https
        volumeMounts:
          - name: config-volume
            mountPath: /etc/nginx/nginx.conf
            subPath: nginx.conf
          - name: tls-certs
            mountPath: /etc/ssl/certs
            readOnly: true
          - name: bundle-storage
            mountPath: /usr/share/nginx/html/bundles
    resources:
      limits:
        cpu: "500m"
        memory: "512Mi"
      requests:
        cpu: "100m"
        memory: "128Mi"
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8443
        scheme: HTTPS
      initialDelaySeconds: 10
      periodSeconds: 5
```

```

volumes:
  - name: config-volume
    configMap:
      name: aegis-bundle-server-config
  - name: tls-certs
    secret:
      secretName: aegis-bundle-server-tls
  - name: bundle-storage
    emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: aegis-bundle-server
  namespace: aegis-system
spec:
  type: ClusterIP
  ports:
    - port: 8443
      targetPort: 8443
      name: https
  selector:
    app: aegis-bundle-server

```

2. Mutating Webhook Configuration (Sidecar Injection)

This configuration registers a Kubernetes Mutating Webhook that automatically injects the SPIRE Workload API volume and the OPA enforcement sidecar into any pod labeled with [aegis.io/inject-sidecar: "true"](#).

YAML

```

apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: aegis-sidecar-injector-webhook
labels:
  app.kubernetes.io/part-of: aegis-control-plane
webhooks:
  - name: inject.sidecar.aegis.io
    rules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE"]
        resources: ["pods"]
        scope: "Namespaced"

```

```

clientConfig:
  service:
    name: aegis-sidecar-injector
    namespace: aegis-system
    path: "/mutate"
    caBundle: "LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCg==" # Base64 encoded
Webhook CA Cert
admissionReviewVersions: ["v1"]
sideEffects: None
timeoutSeconds: 5
failurePolicy: Fail
namespaceSelector:
  matchExpressions:
    - key: aegis.io/plane
      operator: In
      values: ["agent-mesh"]
objectSelector:
  matchLabels:
    aegis.io/inject-sidecar: "true"

```

3. Helm **values.yaml** for Agent Mesh Deployment

This chart value specification configures the automated injection of SPIFFE socket mounts, OPA policy sidecars, and Vector log collectors for target agent workloads.

YAML

```

global:
  environment: production
  trustDomain: swarm.internal

# Sidecar Injection Configuration
sidecarInjector:
  replicaCount: 2
  image:
    repository: aegis-registry.internal/control-plane/injector
    tag: v1.4.0
    pullPolicy: IfNotPresent
  resources:
    limits:
      cpu: 200m
      memory: 256Mi
    requests:
      cpu: 50m
      memory: 64Mi

```

Configuration injected into injected Agent Sidecars

injectedSidecar:

opa:

image: openpolicyagent/opa:0.62.0-envoy

bundleServerUrl:

"https://aegis-bundle-server.aegis-system.svc.cluster.local:8443/v1/bundles/agent-mesh-policy.tar.gz"

pollingIntervalSeconds: 15

verificationPublicKey: |

-----BEGIN PUBLIC KEY-----

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAuZ...

-----END PUBLIC KEY-----

resources:

limits:

cpu: 500m

memory: 512Mi

requests:

cpu: 100m

memory: 128Mi

spire:

socketPath: /run/spire/sockets/agent.sock

hostPathSocket: /var/run/spire/sockets/agent.sock

vector:

enabled: true

image: timberio/vector:0.36.0-alpine

httpIngressPort: 8686

s3SinkBucket: "agent-security-audit-lake"

resources:

limits:

cpu: 300m

memory: 256Mi

requests:

cpu: 50m

memory: 64Mi

Workload Pod Defaults

podAnnotations:

prometheus.io/scrape: "true"

prometheus.io/port: "8888"

Go code for the Mutating Webhook HTTP server that injects the SPIRE volume and OPA sidecar into agent pods.

```
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "log"
    "net/http"
    "os"

    admissionv1 "k8s.io/api/admission/v1"
    corev1 "k8s.io/api/core/v1"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/runtime"
    "k8s.io/apimachinery/pkg/runtime/serializer"
)

var (
    runtimeScheme = runtime.NewScheme()
    codecs        = serializer.NewCodecFactory(runtimeScheme)
    deserializer  = codecs.UniversalDeserializer()
)

func init() {
    _ = corev1.AddToScheme(runtimeScheme)
    _ = admissionv1.AddToScheme(runtimeScheme)
}

type patchOperation struct {
    Op   string `json:"op"`
    Path string `json:"path"`
    Value interface{} `json:"value,omitempty"`
}

type WebhookServer struct {
    server *http.Server
}

func main() {
    port := os.Getenv("PORT")
    if port == "" {
```



```

        port = "8443"
    }

    mux := http.NewServeMux()
    mux.HandleFunc("/mutate", handleMutate)
    mux.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
        _, _ = w.Write([]byte("ok"))
    })

    server := &http.Server{
        Addr:    fmt.Sprintf(":%s", port),
        Handler: mux,
    }

    log.Printf("Starting AEGIS Mutating Webhook Server on port %s...", port)
    certFile := "/etc/webhook/certs/tls.crt"
    keyFile := "/etc/webhook/certs/tls.key"

    if err := server.ListenAndServeTLS(certFile, keyFile); err != nil {
        log.Fatalf("Failed to listen and serve webhook server: %v", err)
    }
}

func handleMutate(w http.ResponseWriter, r *http.Request) {
    var body []byte
    if r.Body != nil {
        if data, err := io.ReadAll(r.Body); err == nil {
            body = data
        }
    }

    if len(body) == 0 {
        http.Error(w, "empty body", http.StatusBadRequest)
        return
    }

    // Verify content type
    contentType := r.Header.Get("Content-Type")
    if contentType != "application/json" {
        http.Error(w, "invalid Content-Type, expect `application/json`",
            http.StatusUnsupportedMediaType)
        return
    }
}

```

```

    admissionReview := admissionv1.AdmissionReview{}
    if _, _, err := deserializer.Decode(body, nil, &admissionReview); err != nil {
        log.Printf("Can't decode body: %v", err)
        http.Error(w, fmt.Sprintf("could not decode admission review: %v", err),
http.StatusBadRequest)
        return
    }

    admissionResponse := mutatePod(admissionReview.Request)
    responseAdmissionReview := admissionv1.AdmissionReview{
        TypeMeta: metav1.TypeMeta{
            APIVersion: "admission.k8s.io/v1",
            Kind:      "AdmissionReview",
        },
    }

    if admissionResponse != nil {
        responseAdmissionReview.Response = admissionResponse
        if admissionReview.Request != nil {
            responseAdmissionReview.Response.UID =
admissionReview.Request.UID
        }
    }

    resp, err := json.Marshal(responseAdmissionReview)
    if err != nil {
        log.Printf("Can't encode response: %v", err)
        http.Error(w, fmt.Sprintf("could not encode response: %v", err),
http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    if _, err := w.Write(resp); err != nil {
        log.Printf("Can't write response: %v", err)
    }
}

func mutatePod(req *admissionv1.AdmissionRequest) *admissionv1.AdmissionResponse {
    var pod corev1.Pod
    if err := json.Unmarshal(req.Object.Raw, &pod); err != nil {
        log.Printf("Could not unmarshal raw object: %v", err)
        return &admissionv1.AdmissionResponse{

```

```

                Result: &metav1.Status{
                    Message: err.Error(),
                },
            },
        }
    }

    // Check if sidecar is already injected
    for _, container := range pod.Spec.Containers {
        if container.Name == "aegis-opa-sidecar" {
            log.Printf("Pod %s/%s already has AEGIS sidecar injected. Skipping.",
req.Namespace, pod.Name)
            return &admissionv1.AdmissionResponse{Allowed: true}
        }
    }

    var patches []patchOperation

    // 1. Add SPIRE Socket HostPath Volume
    socketType := corev1.HostPathSocket
    spireVolume := corev1.Volume{
        Name: "spire-agent-socket",
        VolumeSource: corev1.VolumeSource{
            HostPath: &corev1.HostPathVolumeSource{
                Path: "/var/run/spire/sockets",
                Type: &socketType,
            },
        },
    }

    if len(pod.Spec.Volumes) == 0 {
        patches = append(patches, patchOperation{
            Op: "add",
            Path: "/spec/volumes",
            Value: []corev1.Volume{spireVolume},
        })
    } else {
        patches = append(patches, patchOperation{
            Op: "add",
            Path: "/spec/volumes/-",
            Value: spireVolume,
        })
    }

    // 2. Mount SPIRE Socket into Existing App Containers
    spireVolumeMount := corev1.VolumeMount{

```

```

        Name: "spire-agent-socket",
        MountPath: "/run/spire/sockets",
        ReadOnly: true,
    }

    for i, container := range pod.Spec.Containers {
        path := fmt.Sprintf("/spec/containers/%d/volumeMounts", i)
        if len(container.VolumeMounts) == 0 {
            patches = append(patches, patchOperation{
                Op: "add",
                Path: path,
                Value: []corev1.VolumeMount{spireVolumeMount},
            })
        } else {
            patches = append(patches, patchOperation{
                Op: "add",
                Path: path + "/-",
                Value: spireVolumeMount,
            })
        }
    }
}

```

```

// 3. Inject OPA Sidecar Container
opaSidecar := corev1.Container{
    Name: "aegis-opa-sidecar",
    Image: "openpolicyagent/opa:0.62.0-envoy",
    Args: []string{
        "run",
        "--server",
        "--config-file=/etc/opa/config.yaml",
        "--addr=0.0.0.0:8181",
        "--diagnostic-addr=0.0.0.0:8282",
    },
    Ports: []corev1.ContainerPort{
        {Name: "opa-http", ContainerPort: 8181},
        {Name: "opa-metrics", ContainerPort: 8282},
    },
    VolumeMounts: []corev1.VolumeMount{
        spireVolumeMount,
    },
}

```

```

patches = append(patches, patchOperation{
    Op: "add",

```

```

        Path: "/spec/containers/-",
        Value: opaSidecar,
    })

    patchBytes, err := json.Marshal(patches)
    if err != nil {
        log.Printf("Could not marshal JSON patch: %v", err)
        return &admissionv1.AdmissionResponse{
            Result: &metav1.Status{
                Message: err.Error(),
            },
        }
    }

    patchType := admissionv1.PatchTypeJSONPatch
    return &admissionv1.AdmissionResponse{
        Allowed: true,
        Patch: patchBytes,
        PatchType: &patchType,
    }
}

```